
Unit 2. Classes and Objects

A **Class** is a blueprint that defines **new data types**. This blueprint or prototype can then be used to create objects. classes **do not occupy memory space**. Classes denote the **collection of properties or methods**(Constructor, Nested Class,Interface etc) that is common to all the objects of that class

For example:

- The animal type **Dog** is a class while a particular dog named **Tommy** is an object of the **Dog** class.
- **Student** is a class while a particular student named **Raman** is an object.

Declaration Syntax:

```
access_modifier class <class_name>
{
    type instance-variable1;
    type instance-variable2;
    // ...
    type instance-variableN;

    type methodname1(parameter-list)
    {
        // body of method
    }
    type methodname2(parameter-list)
    {
        // body of method
    }
}
```

Example:

Here is a class called **Box** that defines three instance variables: **width**, **height**, and **depth** and Methods **Init()** and **Display()**.

```
class Box {
    int Width;
    int Height;
    int Depth;

    void init()
    {
        Width = 5;
        Height = 3;
    }
}
```

```

        Depth = 2;
    }
    void Display()
    {
        System.out.println(Width);
        System.out.println(Height);
        System.out.println(Depth);
    }

    public static void main(String args[])
    {
        Box B1= new Box();
        B1.init();
        B1.Display();
    }
}

```

To actually create a **Box** object, you will use a statement like the following:

```
Box B1 = new Box(); // create a Box object called B1
```

- After this statement executes, **B1** will be an instance of **Box**. Thus, it will have “physical” reality.
- Each time you create an instance of a class, you are creating an object that contains its own copy of each instance variable defined by the class.
- Thus, every **Box** object will contain its own copies of the instance variables **width**, **height**, and **depth**. To access these variables and methods, you will use the dot (.) operator.

when you create a class, you are creating a new data type. You can use this type to declare objects of that type. However, **obtaining objects of a class** is a **two-step process**.

- First, you must **declare a variable of the class type**. This variable does not define an object. Instead, it is simply a variable that can refer to an object.

Example: Box B1

- Second, you must obtain an **actual, physical copy of the object** and assign it to that variable. You can do this **using the new** operator. The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by **new**.

Example: `Box B1= new Box();`

`Box mybox = new Box();`

Or

```
Box mybox;           // declare reference to object
mybox = new Box();    // allocate a Box object
```

ASSIGNING OBJECT REFERENCE VARIABLES

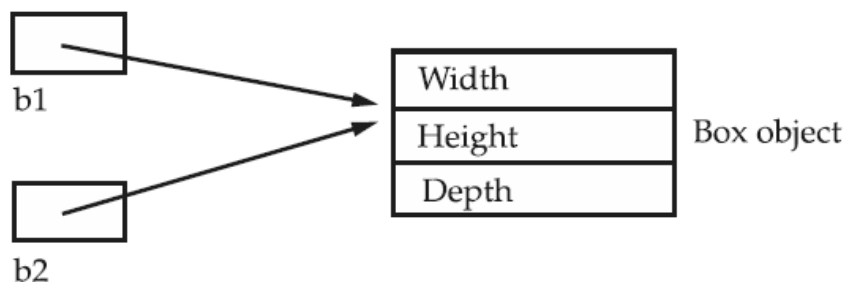
Object reference variables act differently than you might expect when an assignment takes place.

```
Box b1 = new Box();
Box b2 = b1;
```

You might think that **b2** is being assigned a reference to a copy of the object referred to by **b1**. That is, you might think that **b1** and **b2** refer to separate and distinct objects. However, this would be wrong. Instead, after this fragment executes, **b1 and b2 will both refer to the same object.**

The assignment of **b1** to **b2** did not allocate any memory or copy any part of the original object. It simply makes **b2** refer to the same object as does **b1**. Thus, any changes made to the object through **b2** will affect the object to which **b1** is referring, since they are the same object.

This situation is depicted here:



Although **b1** and **b2** both refer to the same object, they are not linked in any other way. For example, a subsequent assignment to **b1** will simply unhook **b1** from the original object without affecting the object or affecting **b2**.

For example:

```
Box b1 = new Box();
```

```
Box b2 = b1;  
// ...
```

```
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

CONSTRUCTORS

To create **special member function that automatically calls itself** when object of that particular class is created. That function is called constructor.

Constructor carries **same name as the class name**. To construct an object and assign values to the object's member. This characteristics differentiate it from other normal function. That means when you found a function with the same name as class name that function is constructor.

Characteristics of Constructor

1. It contain **same name** as class name.
2. It is called **automatically** when object of a particular class is created.
3. It contain **no return** statement and no return type **not even void**.
4. It **cannot** be **invoked** or called explicitly or manually.

Example :

```
/* Here, Box uses a constructor to initialize the  
dimensions of a box.  
*/  
class Box  
{  
    double width;  
    double height;  
    double depth;  
  
    // This is the constructor for Box.  
    Box()  
    {  
        System.out.println("Constructing Box");  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
    double volume()
```

```

        {
            return width * height * depth;
        }
    }

class BoxDemo
{
    public static void main(String args[])
    {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;
        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);
        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}

```

THE THIS KEYWORD

This is a **reference variable** that refers to the current object. To allow this, Java defines the **this** keyword. **this can be used inside any method to refer to the current object.** That is, **this** is always a reference to the object on which the method was invoked. You can use **this** anywhere a reference to an object of the current class' type is permitted. To better understand what **this** refers to, consider the following version of **Box()**:

```

Box(double width, double height, double depth)
{
    this.width = width;
    this.height = height;
    this.depth = depth;
}

```

This version of **Box()** operates exactly like the earlier version. The use of **this** is redundant, but perfectly correct. Inside **Box()**, **this** will always refer to the

invoking object. While it is redundant in this case, **this** is useful in other contexts, one of which is explained in the next section.

Example:

```
class Student
{
    int rollNo;
    String name;
    float fee;
    Student(int rollNo,String name,float fee)//constructor
    {
        this.rollNo=rollNo;
        this.name=name;
        this.fee=fee;
    }
    void display()
    {
        System.out.println(rollNo+" "+name+" "+fee);
    }
}
class TestThis1{
    public static void main(String args[]){
        Student s1=new Student(111,"ankit",5000f);
        Student s2=new Student(112,"sumit",6000f);
        s1.display();
        s2.display();
    }
}
```

Output

111 ankit 5000.0

112 sumit 6000.0

Note : if “this” keyword is not used. Object reference is not generated so variable does not got any value as parameter from object. So output will be

0 null 0.0

0 null 0.0

[GARBAGE COLLECTION](#)

Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation.

In some languages, such as C++, dynamically allocated objects must be manually released by use of a **delete** operator.

Java handles deallocation automatically. The technique that accomplishes this is called garbage collection. It works like **this: when no references** to an object exist, that object is assumed to be **no longer needed**, and the memory occupied by the object can be reclaimed. There is **no explicit need** to destroy objects as in C++.

Garbage collection only **occurs during the execution** of your program. It will not occur simply because one or more objects exist that are no longer used.

We can do using

1) By nulling a reference:

```
Employee e=new Employee();  
e=null;
```

2) By assigning a reference to another:

```
Employee e1=new Employee();  
Employee e2=new Employee();  
e1=e2;//the first object referred by e1 is available for garbage collection
```

THE FINALIZE() METHOD (DESTRUCTOR)

Sometimes an object will need to perform some action when it is **destroyed** or **to perform cleanup processing**. For example, if an object is holding some **non-Java resource** such as a file handle or window character font, then you might want to make sure these resources are **freed before an object is destroyed**. To handle such situations, Java provides a mechanism called **finalization**.

By using finalization, you can define specific actions that will occur when an object is **just about** to be reclaimed **by the garbage collector**. To add a finalizer to a class, you simply define the **finalize()** method. **The Java run time calls that method whenever it is about to recycle an object of that class.**

Inside the **finalize()** method you will specify those actions that must be performed before an object is destroyed. The **garbage collector runs periodically, checking for objects that are no longer referenced by any running state** or indirectly through other referenced objects. Right before an asset is freed, the Java run time calls the **finalize()** method on the object.

The **finalize()** method has this general form:

```
protected void finalize( )  
{  
    // finalization code here
```

```
}
```

Here, the keyword **protected** is a specifier that prevents access to **finalize()** by code defined outside its class. It is important to understand that **finalize()** is only called just **prior to garbage collection**. It is not called when an object goes out-of-scope, for example.

This means that you cannot know when—or even if—**finalize()** will be executed. Therefore, your program should provide other means of releasing system resources, etc., used by the object. It must not rely on **finalize()** for normal program operation.

ACCESS CONTROLS

Access modifiers help to restrict the **scope** of a class, constructor, variable, method, or data member. It provides **security, accessibility**, etc to the user depending upon the access modifier used with the element. Let us learn about Java Access Modifiers, their types, and the uses of access modifiers in this article.

Types of Access Modifiers in Java

There are four types of access modifiers available in Java:

1. Default – No keyword required
2. Private
3. Protected
4. Public

1. Default Access Modifier

When **no access modifier is specified** for a class, method, or data member – It is said to be having the **default** access modifier by default. The data members, classes, or methods that are not declared using any access modifiers i.e. having default access modifiers are accessible **only within the same package**.

Program 1:

```
// Java program to illustrate default modifier
package p1;
// Class stud is having Default access modifier
class stud
{
    void display()
    {
        System.out.println("Hello World!");
    }
}
```


Program 2:

```
// Java program to illustrate error while
// using class from different package with
// default modifier
package p2;
import p1.*;
// This class is having default access modifier
class studnew
{
    public static void main(String args[])
    {
        // Accessing class stud from package p1
        stud obj = new stud();
        obj.display();
    }
}
```

Output:

Compile time error

2. Private Access Modifier

The private access modifier is specified using the keyword **private**. The methods or data members declared as private are accessible only **within the class** in which they are declared.

- Any other **class of the same package will not be able to access** these members.
- Top-level classes or interfaces can not be declared as private because
 - private means “only visible within the enclosing class”.
 - protected means “only visible within the enclosing class and any subclasses”

Hence these modifiers in terms of application to classes, apply only to nested classes and not on top-level classes

In this example, we will create two classes A and B within the same package p1. We will declare a method in class A as private and try to access this method from class B and see the result.

```
// Java program to illustrate error while
// using class from different package with
// private modifier
package p1;
```

```

class A
{
    private void display()
    {
        System.out.println("Hello students");
    }
}

class B
{
    public static void main(String args[])
    {
        A obj = new A();
        // Trying to access private method
        // of another class
        obj.display();
    }
}

```

Output:

error: display() has private access in A
obj.display();

3. Protected Access Modifier

The protected access modifier is specified using the keyword **protected**.

The methods or data members declared as protected are **accessible within the same package or subclasses in different packages**.

In this example, we will create two packages p1 and p2. Class A in p1 is made public, to access it in p2. The method display in class A is protected and class B is inherited from class A and this protected method is then accessed by creating an object of class B.

Program 1:

```

// Java program to illustrate
// protected modifier
package p1;

// Class A
public class A
{

```

```
protected void display()
{
    System.out.println("Hello students ");
}
}
```

Program 2:

```
// Java program to illustrate
// protected modifier
package p2;
import p1.*; // importing all classes in package p1

// Class B is subclass of A
class B extends A
{
    public static void main(String args[])
    {
        B obj = new B();
        obj.display();
    }
}
```

Output:

Hello students

4.Public Access modifier

The public access modifier is specified using the keyword **public**.

- The public access modifier has the **widest scope** among all other access modifiers.
- Classes, methods, or data members that are declared as public are **accessible from everywhere** in the program. There is no restriction on the scope of public data members.

Program 1:

```
// Java program to illustrate
// public modifier
package p1;
public class A
{
```

```

public void display()
{
    System.out.println("Hello students ");
}
}

```

Program 2:

```

package p2;
import p1.*;
class B {
    public static void main(String args[])
    {
        A obj = new A();
        obj.display();
    }
}

```

Output:

Hello students

Important Points:

- If other programmers use your class, try to use the most restrictive access level that makes sense for a particular member. Use private unless you have a good reason not to.
- Avoid public fields except for constants.

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

INHERITANCE IN JAVA

It is the mechanism in Java by which **one class is allowed to inherit the features(fields and methods) of another class**. In Java, Inheritance means creating new classes based on existing ones. A class that inherits from another class can reuse the methods and fields of that class. In addition, you can add new fields and methods to your current class as well.

How to Use Inheritance in Java?

The **extends** keyword is used for inheritance in Java. Using the extends keyword indicates you are derived from an existing class. In other words, “extends” refers to increased functionality.

Syntax :

```
class derived-class extends base-class
{
    //methods and fields
}
```

example of inheritance, class Employee is a base class, class Engineer is a derived class that extends the Employee class and class Test is a driver class to run the program.

// Java Program to illustrate Inheritance (concise)

```
import java.io.*;
```

// Base or Super Class

```
class Employee {
    int salary = 60000;
}
```

// Inherited or Sub Class

```
class Engineer extends Employee {
    int benefits = 10000;
}
```

// Driver Class

```
class Gfg {
    public static void main(String args[])
    {
        Engineer E1 = new Engineer();
        System.out.println("Salary : " + E1.salary
                           + "\nBenefits : " + E1.benefits);
    }
}
```

Output

Salary : 60000

Benefits : 10000

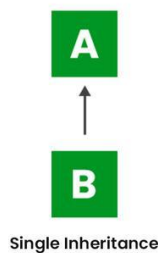
Java Inheritance Types

Below are the different types of inheritance which are supported by Java.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

1. Single Inheritance

In single inheritance, subclasses inherit the features of one superclass. In the image below, class A serves as a base class for the derived class B.



Single inheritance

```
// Java program to illustrate the
// concept of single inheritance
import java.io.*;
import java.lang.*;
import java.util.*;

// Parent class
class One {
    public void print_bca()
    {
        System.out.println("Bca");
    }
}

class Two extends One {
    public void print_for()
    { System.out.println("for");
    }
}

// Driver class
public class Main {
```

```

// Main function
public static void main(String[] args)
{
    Two g = new Two();
    g.print_bca();
    g.print_for();
    g.print_bca();
}
}

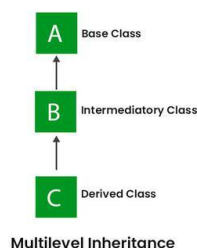
```

Output

Bca
for
Bca

2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the [grandparent's members](#).



Multilevel Inheritance

```

// Java program to illustrate the
// concept of Multilevel inheritance
import java.io.*;
import java.lang.*;
import java.util.*;

class One {
    public void print_geek()
    {
        System.out.println("Geeks");
    }
}

```

```

}

class Two extends One {
    public void print_for() { System.out.println("for"); }
}

class Three extends Two {
    public void print_geek()
    {
        System.out.println("Geeks");
    }
}

// Drived class
class Main {
    public static void main(String[] args)
    {
        Three g = new Three();
        g.print_geek();
        g.print_for();
        g.print_geek();
    }
}

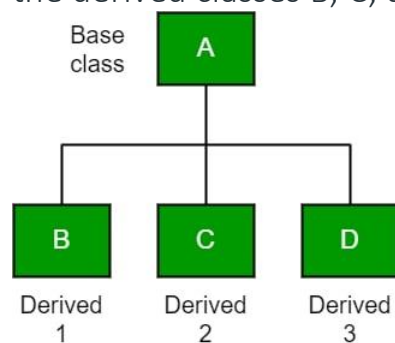
```

Output

Geeks
for
Geeks

3. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.




```
// Java program to illustrate the
// concept of Hierarchical inheritance

class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

Output

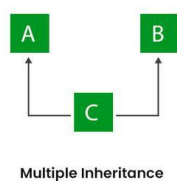
Class A

Class B

Class A
Class C
Class A
Class D

4. Multiple Inheritance (Through Interfaces)

In [Multiple inheritances](#), one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support [multiple inheritances](#) with classes. In Java, we can achieve multiple inheritances only through [Interfaces](#). In the image below, Class C is derived from interfaces A and B.



Multiple Inheritance

```
// Java program to illustrate the
// concept of Multiple inheritance
import java.io.*;
import java.lang.*;
import java.util.*;

interface One {
    public void print_geek();
}

interface Two {
    public void print_for();
}

interface Three extends One, Two {
    public void print_geek();
}

class Child implements Three {
    public void print_geek()
    {
        System.out.println("Geeks");
    }
}
```

```

    public void print_for() { System.out.println("for"); }
}

// Drived class
public class Main {
    public static void main(String[] args)
    {
        Child c = new Child();
        c.print_geek();
        c.print_for();
        c.print_geek();
    }
}

```

Output

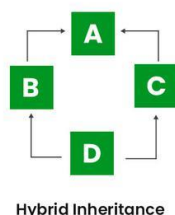
Geeks
for
Geeks

5. Hybrid Inheritance

It is a **mix of two or more of the above types of inheritance**. Since Java doesn't support multiple inheritances with classes, hybrid inheritance involving multiple inheritance is also not possible with classes.

In Java, we can achieve hybrid inheritance only through [Interfaces](#) if we want to involve multiple inheritance to implement Hybrid inheritance.

However, it is important to note that Hybrid inheritance does not necessarily require the use of Multiple Inheritance exclusively. It can be achieved through a combination of Multilevel Inheritance and Hierarchical Inheritance with classes, Hierarchical and Single Inheritance with classes. Therefore, it is indeed possible to implement Hybrid inheritance using classes alone, without relying on multiple inheritance type.



Hybrid Inheritance

Java IS-A type of Relationship

IS-A is a way of saying: This object is a type of that object. Let us see how the extends keyword is used to achieve inheritance.

```
public class SolarSystem {  
}  
public class Earth extends SolarSystem {  
}  
public class Mars extends SolarSystem {  
}  
public class Moon extends Earth {  
}
```

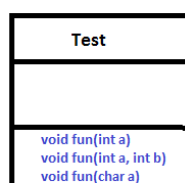
Now, based on the above example, in Object-Oriented terms, the following are true:-

- SolarSystem is the superclass of Earth class.
- SolarSystem is the superclass of Mars class.
- Earth and Mars are subclasses of SolarSystem class.
- Moon is the subclass of both Earth and SolarSystem classes.

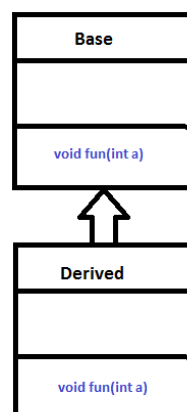
Polymorphism in Java

Polymorphism allows us **to perform a single action in different ways**. In other words, polymorphism allows you to define one interface and have multiple implementations. The word “poly” means many and “morphs” means forms two types:

1. Compile-time Polymorphism(Method Overloading)
2. Runtime Polymorphism(Method overriding/ Dynamic Method Dispatch)



Overloading



Overriding

1)Compile-Time Polymorphism in Java(overloading)

It is also known as static polymorphism. This type of polymorphism

1. function overloading
2. operator overloading(java doesn't support)

In Function overloading, multiple functions can have the same name but with different parameter lists. The compiler will decide which function to call based on the number and types of arguments passed to the function.

Example:

```
// Java Program for Method overloading
// By using Different Types of Arguments

// Class 1
// Helper class
class Helper {

    // Method with 2 integer parameters
    static int Multiply(int a, int b)
    {
        // Returns product of integer numbers
        return a * b;
    }

    // Method 2
    // With same name but with 2 double parameters
    static double Multiply(double a, double b)
    {
        // Returns product of double numbers
        return a * b;
    }

    // Method 3
    // // Multiplication of 3 numbers
    static int Multiply(int a, int b, int c)
    {
        // Return product
        return a * b * c;
    }
}

// Class 2
// Main class
class GFG {
    // Main driver method
    public static void main(String[] args)
```

```

    {
        // Calling method by passing
        // input as in arguments
        System.out.println(helper.multiply(2, 4));
        System.out.println(helper.multiply(5.5, 6.3));
        System.out.println(helper.multiply(2, 4, 4));
    }
}

```

Output

```

8
34.65
32

```

2) Runtime Polymorphism

It occurs when a derived class(SUB CLASS) has a definition for one of the member functions of the base class(SUPER CLASS).

Example:

```

// Java Program for Method Overriding
// Class 1
// Helper class
class Parent {
    // Method of parent class
    void Print()
    {
        // Print statement
        System.out.println("parent class");
    }
}
// Class 2
// Helper class
class subclass1 extends Parent {
    // Method
    void Print()
    {
        System.out.println("subclass1");
    }
}
// Class 3
// Helper class
class subclass2 extends Parent {
    // Method
    void Print()

```

```

        {      // Print statement
            System.out.println("subclass2");
        }
    }
// Class 4
// Main class
class GFG {
    // Main driver method
    public static void main(String[] args)
    {
        // Creating object of class 1
        Parent a;

        // Now we will be calling print methods
        // inside main() method

        a = new subclass1();
        a.Print();

        a = new subclass2();
        a.Print();
    }
}

```

Output

subclass1

subclass2

Here in this program, When an object of a child class is created, then the method inside the child class is called. This is because The method in the parent class is overridden by the child class.

Since The method is overridden, This method has **more priority than the parent method inside the child class**. So, the body inside the child class is executed.

Advantages of Polymorphism in Java

1. Increases code reusability by allowing objects of different classes to be treated as objects of a **common class**.
2. Improves readability and maintainability of code by **reducing the amount of code** that needs to be written and maintained.
3. Supports dynamic binding, enabling the **correct method to be called at runtime**, based on the actual class of the object.

Disadvantages of Polymorphism in Java

1. Can make it more difficult to understand the behavior of an object, especially if the **code is complex**.
2. This may lead to performance issues, as polymorphic behavior may require **additional computations at runtime**.

Data encapsulation

Encapsulation is defined as the **wrapping up of data under a single unit**. It is the mechanism **that binds together code and the data it manipulates**.

Another way to think about encapsulation is, that it is a **protective shield that prevents the data from being accessed by the code outside this shield**.

- Technically in encapsulation, the variables or data of a **class is hidden from any other class and can be accessed only through any member function** of its own class in which it is declared.
- As in encapsulation, the data in a class is hidden from other classes using the data hiding concept which is achieved by making **the members or methods of a class private**, and the class is **exposed to the end-user or the world without providing any details** behind implementation using the abstraction concept, so it is also known as a **combination of data-hiding and abstraction**.
- Encapsulation can be achieved by **Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables**. It is more defined with the **setter and getter** method.

Example:

```
// Java Program to demonstrate
// Java Encapsulation

class Name {
    // Private is using to hide the data
    private int age;//data hiding

    // getter
    public int getAge()
    {
        return age;
    }

    // setter
    public void setAge(int age)
    { this.age = age;
    }
```



```

}

// Driver Class
class GFG {
    // main function
    public static void main(String[] args)
    {
        Name n1 = new Name();
        n1.setAge(19);
        System.out.println("The age of the person is: "
                           + n1.getAge());
    }
}

```

Output

The age of the person is: 19

Advantages of Encapsulation

- **Data Hiding:** it is a way of restricting the access of our data members by hiding the implementation details. Encapsulation also provides a way for data hiding. The user will have no idea about the inner implementation of the class. **It will not be visible to the user how the class is storing values in the variables. The user will only know that we are passing the values to a setter method and variables are getting initialized with that value.**
- **Increased Flexibility:** We can make the variables of the class read-only or write-only depending on our requirements. **If we wish to make the variables read-only then we have to omit the setter methods like setName(), setAge(), etc. from the above program or if we wish to make the variables write-only then we have to omit the get methods like getName(), getAge(), etc. from the above program**

Disadvantages of Encapsulation in Java

- Can lead to increased complexity, especially if not used properly.
- Can make it more difficult to understand how the system works.

super keywords

In java, **super** keyword is used to access methods of the **parent class** while **this** is used to access methods of the **current class**.

[super](#) keyword

1. **super** is a reserved keyword in java i.e, we can't use it as an identifier.
2. **super** is used to refer **super-class's** instance as well as static members.
3. **super** is also used to invoke **super-class's** method or constructor.

4. The most common use of **super** keyword is that it **eliminates the confusion between the superclasses and subclasses** that have methods with **same name**.

super can be used in various contexts as given below:

- 1) it can be used to refer immediate parent class instance variable (use of **super** with variable)

We can use **super** keyword to access the data member or field of parent class. It is used if **parent class and child class have same fields**.

```
class A
{
    int a=10; //instance variable of class A
}
class B extends A
{int a=20;// instance variable of class B
    void disp(int a)
    {
        System.out.println(a);//it reference parameter value
        System.out.println(this.a);//it use reference variable(this) current
class instance variable
        System.out.println(super.a);//super use parent class instance variable.
    }
}
class c
{public static void main(String arg[])
    {B b1=new B();
        b1.disp(30);
    }
}
```

Output:

30

20

10

- 2) It can be used to refer immediate parent class method (use of **super** with method)

The **super** keyword can also be used to invoke parent class method. It should be used if subclass contains the **same method as parent class**. In other words, it is used if method is **overridden**.

```
class Animal
```

```

{
    void eat()
    {
        System.out.println("eating...");
    }
}
class Dog extends Animal
{
    void eat(){System.out.println("eating bread...");}
    void bark(){System.out.println("barking...");}
    void work()
    {
        super.eat();
        bark();
    }
}
class TestSuper2
{
    public static void main(String args[])
    {
        Dog d=new Dog();
        d.work();
    }
}

```

Output:

Eating...

Barking...

3) It can be used to refer immediate parent class constructor. (use of super with constructors)

A subclass can call a constructor method defined by its superclass by use of the following form of **super**:

```
super(parameter-list);
```

Here, parameter-list specifies any parameters needed by the constructor in the superclass. **super()** must always be the first statement executed inside a subclass' constructor.

Example :

```
class A
```

```

{ A(int x)
    {System.out.print(x);
    }
}
class B extends A
{
    B()
    { super(8); //use constructor of parent(super) class
      System.out.print("hi");
    }
}
class c
{
    public static void main(String arg[])
    {new B();
    }
}

```

Notice that **super(8)** not written **than error of passing argument constructor is occurs**. because parameter is not specify at the time of object creation.

Similarities in this and super

- We can use this as well as super **anywhere except static area**. Example of this is already shown above where we use this as well as super inside public static void main(String[]args) hence we get Compile Time Error since cannot use them inside static area.
- We can use this as well as super **any number of times in a program**.
- Both are **non-static** keyword.

Final keyword

The final keyword is a non-access modifier **used for classes, attributes and methods, which makes them non-changeable** (impossible to inherit or override)

The keyword **final** has three uses.

1. Variable(To make constant)
2. Method(Prevent method overriding)
3. Class(Prevent method Inheritance)

1. Using final to make constant

To change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```

class A{

```

```

final int x=9;//final variable
    void run()
    {
        x=4;
    }
public static void main(String args[])
{
    A obj=new A();
    obj.run();
}
} //end of class

```

Output

Output:Compile Time Error

2.Using final to Prevent Overriding

To disallow a method from being overridden, specify **final** as a modifier at the start of its declaration. Methods declared as **final** cannot be overridden. The following fragment illustrates **final**:

```

class A {
    final void meth() {
        System.out.println("This is a final method.");
    }
}
class B extends A {
    void meth() { // ERROR! Can't override.
        System.out.println("Illegal!");
    }
}

```

Because **meth()** is declared as **final**, it cannot be overridden in **B**. If you attempt to do so, a compile-time error will result. Methods declared as **final** can sometimes provide a performance enhancement: The compiler is free to *inline* calls to them because it “knows” they will not be overridden by a subclass. When a small **final** method is called, often the Java compiler can copy the bytecode for the subroutine directly inline with the compiled code of the calling method, thus eliminating the costly overhead associated with a method call. Inlining is only an option with **final** methods.

Normally, Java resolves calls to methods dynamically, at run time. This is called *late binding*. However, since **final** methods cannot be overridden, a call to one can be resolved at compile time. This is called *early binding*.

3. Using final to Prevent Inheritance

Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with **final**. Declaring a class as **final** implicitly declares all of its methods as **final**, too. As you might expect, it is illegal to declare a class as both **abstract** and **final** since an abstract class is incomplete by itself and relies upon its subclasses to provide complete implementations. Here is an example of a **final** class:

```
final class A {  
    // ...  
}  
// The following class is illegal.  
class B extends A { // ERROR! Can't subclass A  
    // ...  
}
```

As the comments imply, it is illegal for **B** to inherit **A** since **A** is declared as **final**.

Static members, static block, static class

Static keyword can be used with **class, variable, method and block**. Static members belong to the class instead of a specific instance, this means if you make a member static, **you can access it without object**.

Static method:

we have a static method `myMethod()`, we can **call this method without any object because when we make a member static it becomes class level**. If we **remove the static keyword and make it non-static** then we must need to create an object of the class in order to call it.

Example:

```
class SimpleStaticExample  
{  
    // This is a static method
```

```

static void myMethod()
{
    System.out.println("myMethod");
}

public static void main(String[] args)
{
    myMethod();
}
}

```

Output:

```
myMethod
```

Static block and variable:

Static block is used for initializing the static variables. This block gets executed when the class is loaded in the memory. A class can have multiple Static blocks, which will execute in the same sequence in which they have been written into the program.

Example:

```

class JavaExample{
    static int num;
    static String mystr;
    static{
        num = 97;
        mystr = "Static keyword in Java";
    }
    public static void main(String args[])
    {
        System.out.println("Value of num: "+num);
        System.out.println("Value of mystr: "+mystr);
    }
}

```

Output:

```

Value of num: 97
Value of mystr: Static keyword in Java

```

Static class

A class can be made **static** only if it is a nested class.

1. Nested static class **doesn't** need reference of Outer class

2. A static class **cannot access non-static** members of the Outer class
example if "int x" is define after "str" variable.it is not accessible
because it is nonstatic.

Example:

```
class JavaExample{
    private static String str = "BeginnersBook";

    //Static class
    static class MyNestedClass{
        //non-static method
        public void disp() {

            /* If you make the str variable of outer class
             * non-static then you will get compilation error
             * because: a nested static class cannot access non-
             * static members of the outer class.
             */
            System.out.println(str);
        }
    }

    public static void main(String args[])
    {
        /* To create instance of nested class we didn't need the outer
         * class instance but for a regular nested class you would need
         * to create an instance of outer class first
         */
        MyNestedClass obj = new MyNestedClass();
        obj.disp();
    }
}
```

Output:

BeginnersBook

Abstract:

Data **abstraction** is the process of hiding certain details and showing only essential information to the user.

Abstraction can be achieved with either **abstract classes** or **interfaces**

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Syntax: *abstract type name(parameter-list);*

- As you can see, no method body is present. Any class that contains **one or more abstract methods** must also be declared abstract.
- To declare a class abstract, you simply use the **abstract** keyword in front of the **class** keyword at the beginning of the class declaration.
- There can **be no objects** of an abstract class.
- An abstract class cannot be directly instantiated with the **new** operator. Such objects would be **useless**, because an abstract class is **not fully defined**. Also, you cannot declare abstract constructors, or abstract static methods.
- Any **subclass of an abstract** class must either implement all of the abstract methods in the **superclass**, or be itself declared **abstract**.

Example:

```
abstract class A
{
    abstract void callme();
    // concrete methods are still allowed in abstract classes
    void callmetoo() {
        System.out.println("This is a concrete method.");
    }
}
class B extends A
{
    void callme() {
        System.out.println("B's implementation of
        callme.");
    }
}
class AbstractDemo
{
    public static void main(String args[])
    {
        B b = new B();
        b.callme();
        b.callmetoo();
    }
}
```

Notice that no objects of class **A** are declared in the program. As mentioned, it is not possible to instantiate an abstract class.

One other point: class **A** implements a concrete method called **callmetoo()**. This is perfectly acceptable. Abstract classes can include as much implementation as they see fit. Although abstract classes cannot be used to instantiate objects, they can be used to create object references, because Java's approach to run-time polymorphism is implemented through the use of superclass references. Thus, it must be possible to create a reference to an abstract class so that it can be used to point to a subclass object.

Interfaces:

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

- The interface in Java is *a mechanism to achieve abstraction*.
- There can be only abstract methods in the Java interface, **not method body**.
- It is **used to achieve abstraction and multiple inheritance in Java**.

Java Interface also **represents the IS-A relationship**.

Declare:

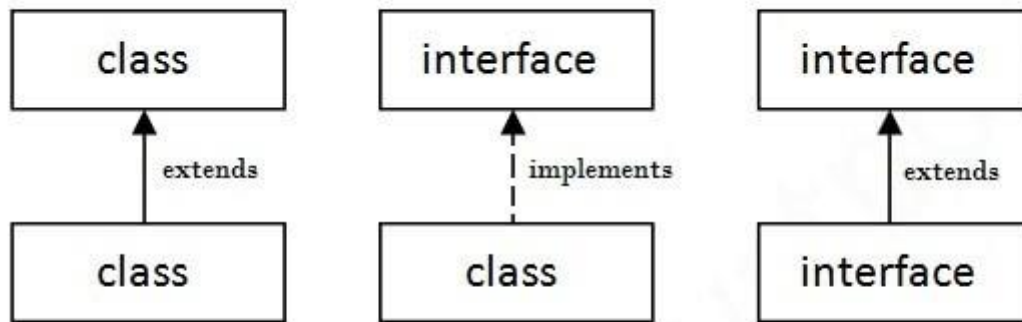
- An interface is declared by using the **interface** keyword.
- It provides total abstraction method; means all the methods in an **interface are declared with the empty body**
- **all the fields(variable) are public, static and final by default.**
- In 8th /9th version of java, we can Create public static,private method and default method.

Syntax:

```
interface <interface_name>
{
    Methods
        (only abstract and public(compiler automatically consider public)
    Fields/variable
        (only public, static and final by default. )
}
```

The relationship between classes and interfaces

As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



Example:

```
interface I1
{void show();//blueprint of method
}
class test implements I1
{ public void show() //public bcz show() in main class is public by default
  {System.out.print("hi");
  }
}
class A
{public static void main(String arg[])
{test t= new test();
t.show();
}
}
```

Extending one interface into another(Making an interface from another interface)

Example:

```
interface I1
{ float pi=3.14;
  Void show();
}
Interface I2 extends I1
{
  Int b=20;
  Void display();
}
```

```
}
```

Deriving an interface from multiple interface: we can also make single interface from multiple interface.

To extend more interface extend into one interface we have to use extend keyword and each interface is separated by comma.

Example:

```
interface I1
{void show1();//blueprint of method
}
interface I2
{void show2();//blueprint of method
}
class test implements I1,I2
{ public void show1() //public bcz show() in main class is public by default
    {System.out.print("hi");
    }
    public void show2() //public bcz show() in main class is public by default
    {System.out.print("hello");
    }

}
class A
{public static void main(String arg[])
{test t= new test();
t.show1();
t.show2();
}
}
```

Implementation Interface:

- Once an interface has been define or more class can implement the interface
- To implement an interface into a class use **implements** keyword
- In one class,you can implement **more than one interface which are separated by comma**.
- The **methods of interface must be declare public** when you implement an interface into class.
- This **type signature of implementing method** must **match** exactly as define in **interface** declaration.
- If you implement an interface in a **class** then you have to make **body of all the methods** which are **declared in interface**.

- If you **not declare body** of all the methods then that **class will abstract**.

Accessing methods through interface references:

The constant value will be available to any class that implements the interface. These values can be used in any methods.

We cannot create object of an interface but we can **create reference of interface**.

Example:

```
interface I1
{
    void show();
}
class A implements I1
{public void show()
    {
        System.out.println("hello");
    }
}
class B implements I1
{    public void show()
    { System.out.println("hi");
    }
}
class InterDemo
{public static void main(String arg[])
    {
        I1 ob; //reference of interface
        A a1=new A(); //object of class A
        ob=a1; //reference connect with object
        ob.show();
        B b1=new B();
        ob=b1;
        ob.show();
    }
}
```

Nested Interface

An interface can have another interface it is known as nested interface

Ex:

```
Interface p1
{void p()
}
```

```

        Interface v1
        {
            Void view();
        }
    }

```

Multiple inheritance (using extends and implements together):

We can both extends and implements together in a single program **First you have to extend the class and then implements the interface into the class. You cannot interchange the sequence.** Whenever you are using “extends” and “implements” into a class then it is a case of multiple inheritance.

Example:

```

interface I1
{
    void show1();
}
interface I2 extends I1
{
    void show2();
}
class demo implements I2
{
    public void show1()
    {
        System.out.print("hi");
    }
    public void show2()
    {
        System.out.print("hello");
    }
}
class A
{public static void main(String arg[])
    {
        demo d=new demo();
        d.show1();
        d.show2();
    }
}

```

Here Interface I2 inherits properties of interface I1. Class demo implements the interface i2. It is override all the method of I1 and I2.

Partial Implementation:

When you are implementing the interface in a class and if you are not implement all the methods of interface then it is known as abstract class.

In this case, we have to declare that class .

Example:

```
interface i1
{
    void method1();
    void method2();
}
abstract class demo implements i1
{
    public void method1()
    {System.out.print("hi");
    }
}
class demo1 extends demo
{public void method2()
    {System.out.print("hello");
    }
    public static void main(String arg[])
    {demo1 d=new demo1();
    d.method1();
    d.method2();
    }
}
```

Here interface I1 implemented by class “demo” but it only override method1.so “demo” class declared as an “abstract class” and “method2” which is not overridden by class “demo” must be overridden in “subclass of demo class demo1”